1 BASICS, TYPES, VARIABLES (Wk1)

- Algorithm = well-defined steps, input -> output, machine independent. Program = code on a machine. Shown by **pseudocode** or **flowchart**.
- Var name: start letter/_; only letters, digits, _; case-sensitive; not a keyword. `my-var`, `my var`, `2x` invalid. Python style = snake_case (wrong style is NOT an error).
- Types: `int` `float` `bool` `str` `NoneType`. `type(x)` gives class. `=` is **assignment** (RHS first), `==` is equality.

```
a, b = 1, 2        a, b, c = c, b, a   # swap
```

- Convert: `int(23.8)`=23 (truncates), `float('45.2')`, `str(123)`='123'. `int('10 plus 20')` and `int('3.5')` -> **ValueError**.
- `input()` always returns **str** -> wrap `int(input(..))`.
- `import math` -> `math.cos(0)`; `from math import cos, pi` -> `cos(0)`; `import math as m`. `import` avoids name clashes. `random.randint(0,99)` includes BOTH ends. `math.sqrt(-1)` -> error (check delta<0 first).
- Built-ins need no import: `print input type len abs min max sum range int float str list tuple set` ... (`abs(-3)`=3, `min(3,1)`=1). Explore docs.python.org.

2 OPERATORS & BOOLEANS

```
+ - * / ** // %   11/3=3.666 11//3=3 11%3=2
2**3=8  2**3**2=512 (right-assoc)  7//2=3
-7//2=-4 (floors!)  -7%3=2  int(-2.7)=-2
```

- `/` always gives float. `//` floors (towards -inf). `%` takes sign of divisor. Digits: `n%10` last digit, `n//10` drop it.
- **Precedence** high->low: `()`, `**`, unary `-`, `* / // %`, `+ -`, comparisons (`< <= == != > >= in`), `not`, `and`, `or`. So `False or True and True` = True; `-2**2` = -4; `6/2*(1+2)` = 9.0 (left to right).
- Comparison: `<= >= == !=`. `'1'==1` False. `True==1`, `False==0`; `True+1`=2, `not False+1` = `not 1` = False.
- **Falsy**: `0`, `0.0`, `''`, `[]`, `()`, `{}`, `set()`, `None`. Everything else truthy.
- `and` / `or` **return an operand, not just True/False**: `True and 0` -> 0; `1 or 0` -> 1; `0 or 'a'` -> 'a'; `'' and 5` -> ''. `not` always gives bool.

TRAP: Short-circuit: `False and (1/0)` -> False (no error); `True or error` -> True. `'a'+3` is TypeError (str+int) but is skipped if never evaluated, e.g. `False and 'a'+3 or 1<2` -> True.

- Strings: `+` join, `*n` repeat, `in` substring, compare lexicographically char by char ('bananb' > 'banana'; uppercase < lowercase). f-string: `f"{x}x tea ${p}"`. `'ab'*2+'c'` = 'ababc'. `str*str` or `str+int` = error.
- Chained/mixed checks: `x in 'aeiou'`, `a < b < c` works, `c not in lst`, `'4' in str(n)` (test digit). `n % 2 == 0` even, `n % 2` truthy if odd.

3 INDEXING & SLICING (strings, lists, tuples)

```
s = 'abcdef'   idx: 0 1 2 3 4 5  /  -6 ... -1
s[start:stop:step] stop EXCLUDED
  defaults: start 0, stop len, step 1
s[1:5:3]='be'  s[::2]='ace'  s[::-1]='fedcba'
s[2:]  s[:2]  s[-2:]  s[3:3]=''  s[4:1]=''
```

TRAP: Single index out of range -> **IndexError** (`s[6]`, `'abc'[3]`). **Slices never error**: out-of-range ends are clipped ('abc'[3:5]='' , 'abcdefg'[1:9]='bcdefg'). Wrong direction ('mangosteen'[-5:-10]) = empty. Step -1 needs start>stop.

- Slice returns same type: tuple slice -> tuple, list slice -> list, str -> str. `s[3:4]` is 1-char STRING, `s[3]` is char.
- Strings are **immutable**: `s[0]='x'` -> TypeError. Make new string instead.
- All indexed sequences (str, list, tuple): `a[i]` `a[i:j]` `len(a)` `min(a)` `max(a)` `x in a` `a+b` `n*a` `a.index(x)` `a.count(x)`.
- `range(stop)`, `range(start,stop)`, `range(start,stop,step)`: stop excluded. `range(0,5,-1)` is EMPTY; `range(5,0,-1)`=5..1. `list(range(..))` to see it.
- `s.isalpha()`, `s.upper()`, `s.lower()`, `s.split()`, `s.rstrip()`, `'-'.join(lst)` return NEW strings (strings never change in place).

4 FUNCTIONS

```
def square(x):       # name(parameters)
    return x * x     # indent = body
square(3)  -> 9       square(3,4) -> TypeError
```

- Function with **no return** (or bare `return`) gives `None`. `print` shows, `return` gives value back; `print(f())` prints None if f only prints. Code after `return` never runs.
- Wrong number of args -> TypeError. Defining a function does NOT run it. Functions may call functions defined later, as long as they exist at call time. Use `return a, b` -> tuple.
- Indentation matters in Python: lines inside def/if/for/while must align. Un-indented line is outside.
- Multiple return paths: if no branch returns, falls off the end -> `None` (see exam Q12: `foo(p2)` prints p then None).
- Abstraction: name a block of code once and reuse it (e.g. `drawTriangle()`).

5 SELECTION (if / elif / else) (Wk2)

```
if cond1:      ...
elif cond2:    ...    # many elif allowed
else:          ...    # optional
```

- Only the **first True** branch runs; the rest are skipped. Order of `elif` matters (grade: test smallest range first, e.g. `<40`, `<45`, `<55`, `<70`, else A). Nested `if` allowed; `if` may have no `else` (nothing printed).
- Conditions can be any value (truthiness): `if lst:` = non-empty; `if not n:`. Note `'0'` and `'False'` (non-empty str) are truthy.
- Multiple separate `if` (not elif) can ALL run.
- Indent errors: `else` must line up with its `if`. A dangling `else` after a nested `if` belongs to the nearest `if` at the same indent.

6 LOOPS (Wk2)

```
for x in <iterable>:    # str,list,tuple,set,
    body                # dict,range,file
while cond:             # re-checked each time
    body
```

- **for**: number of rounds known (`range(n)`, each item). **while**: unknown/changing count (until guess right, input 'bye'). `for i in range(0,N)` = the common `while a<N` pattern.
- `break` leaves the **innermost** loop immediately; `continue` skips to next iteration. In nested loops, `break` in inner does NOT stop outer (exam Q28: `break` in outer loop stops all at first space).
- Loop var stays after loop. `while True:` runs forever unless `break`/`return`. Forgetting to update the variable = infinite loop. Off-by-one: `<=` vs `<` (factorial: `counter <= n`).
- **Loop types**: run exactly N; run any number of times; at most N (early exit).

```
# check ALL true: return False at 1st failure
for c in s:
    if not c.isalpha(): return False
return True        # after loop only!
# check ANY: return True at 1st hit, else False
```

```
# accumulate: init -> update -> return
res = 0
for d in str(n): res += int(d)
# digits: while n>0: res+=n%10; n//=10
count = 0; x = 50
while x >= 1:          # x: 50,12,3,0
    x //= 4; count += 1  # count ends = 3
```

```
# nested: inner runs fully each outer pass
for i in range(1,4):
    for j in range(i): print('#'*j)
```

```
# trace table: write var values per pass
balance=100
for i in range(15):
    if i > balance: break
    balance -= i       # 100-(0+1+...+13)=9
```

7 SCOPE & CALL STACK (W3a)

- **Global** = defined outside functions (visible everywhere for reading). **Local** = created by assignment inside function, or a parameter; dies when the function returns.
- Golden rule: `x = ...` anywhere in a function body makes `x` **local to that whole function**. Local `x` hides the global; the global is unchanged.

```
x = 0
def f(): x = 999; print(x)   # 999
f(); print(x)                # 0
```

TRAP: `def f(): print(x); x = 999` -> **error** (UnboundLocalError): `x=999` makes x local, so the first print uses x before it exists.

- Only reading: `def f(): print(x)` finds the global x (no error). To modify a global use `global x` inside the function (then no local x is created).
- Parameters are local. Each function has its own scope (like each 'Taylor' in own class). A function sees **its own locals and globals**, NOT the locals of the function that called it (exam Q26: `bar` uses global `n=10`, not foo's `n=100`).
- Reassigning a parameter inside a function never changes the caller's variable (see sec 10 for lists).
- Convention: globals only as UPPERCASE constants; use parameters + return instead.
- **Call stack**: LIFO (first in, last out). Calling pushes a frame; returning pops. Prints 'Entering p1, p2, p3 ... line before return p3, p2, p1'.
- Nested `def` inside a function is local to it (can't be called outside).

8 RECURSION (W3a, W3c)

- A function that **calls itself** on a smaller version of the problem. Needs: (1) **base case** (stop), (2) recursive step that **shrinks** the problem toward the base. Missing either -> infinite recursion (RecursionError).

```
def fact(n):
    if n == 0: return 1
    return n * fact(n-1)
def fib(n):   # f(0)=f(1)=1 (slide)
    if n <= 1: return 1
    return fib(n-1) + fib(n-2)
def sumdig(n):
    if n == 0: return 0
    return n % 10 + sumdig(n // 10)
def rev(s):
    if s == '': return ''
    return rev(s[1:]) + s[0]   # abcde->edcba
```

- Naive `fib` recomputes the same values (exponential calls). For big n (n>1000) use iteration.
- **Counting calls / prints** (like `pascal(r,c)`): draw the call tree; each call incl. base cases counts as a call. Check base cases in the order written: `r<c` first, then `r<=1 or c<=1`.
- **Iteration <-> Recursion**: initial value = base case; accumulated result = what the call returns; update step (`n=n//10`) = argument of next call; loop condition (`n>0`) = opposite of base case. Recursion needs no temp result variables (the stack holds them).
- Iteration pattern: init answer to 'nothing' (0, 1, '', []), accumulate one-by-one, return. Recursion pattern: base case + reduce one-by-one, combine with `+` / `*`.
- Sum to k terms (Taylor/sin): `S(k) = S(k-1) + kth term`; base case = the first term / k=0.

9 LIST / TUPLE / SET / DICT (Wk4a)

Sequences: str, list, tuple (indexed, ordered). Set and dict are **not indexed**. All are iterable (can go after `in` in `for`).

```
L = [1,'a',[2,3]]  list: mutable, mixed, [ ]
T = (1,'a',[2,3])  tuple: immutable, ( )
S = {1,2,3}        set: unordered, unique
D = {'a':1,'b':2}  dict: key:value pairs
```

- **Singleton**: `(5,)` is tuple (comma!), `(5)` is int 5, `[5]` is list. `{}` is an empty **dict**; empty set = `set()`. `()` empty tuple. `((2,3),)[-1]` = (2,3).
- **List methods** (in place, return None!): `L.append(x)` add one item at end; `L.extend(it)` or `L += it` add each item; `L.insert(i,x)`; `L.pop()` / `L.pop(i)` removes & returns; `L.remove(x)` removes FIRST x only, **ValueError** if absent; `L.sort()` ascending in place (returns None), `sorted(L)` returns new list; `L.reverse()`; `del L[i]`; `L.copy()`; `L[i]=v`; `L[1:]=[..]` slice assign (length can change).

TRAP: `L.append([4,5])` nests -> `[1,[4,5]]`; `L += [4,5]` / `extend` flattens -> `[1,4,5]`. `x = L.sort()` makes x = None. `[1,2]+(3,)` TypeError. `tuple +=` builds a NEW tuple.

- **Tuple**: no append/remove/assign (TypeError), but a **mutable item inside can change**:

`t=([1,2],3); t[0][0]=9` OK -> `([9,2],3)`. Tuple holds the same object (see t1/t2 example: changing list shows in tuple).

- Culture: list = many same-type items (200 names); tuple = few mixed-type fields (a record, an (x,y)). Breaking it is not a syntax error.
- **Convert**: `list('123')`=['1','2','3'], `tuple(..)`, `set('bananas')`, `str(list)` gives text "['1',..]"; `''.join(['a','b'])`='ab' (list of str only); `'a b'.split()`=['a','b']; `list(range(3))`.
- **List comprehension** `[expr for x in it if cond]`: `[i*i for i in range(1,11)]`, odd `[i for i in range(1,100,2)]`, `[c+d for c in s for d in t]`. Primes < 50: non-primes = `[j for i in range(2,8) for j in range(2*i,50,i)]`; primes = numbers not in it.
- **Set**: dupes removed automatically (`len({5,3,5})`=2); no index, no slice, no `+`, no `*`. `a|b` union, `a&b` intersection, `a-b` difference, `a^b` symmetric diff. `s.add(x)`, `s.remove(x)` (KeyError if absent), `s.discard(x)` (no error). `x in s`, `len(s)`. Set of a string = unique chars. Set items must be immutable (no list inside).
- **Dict**: `d[k]` read (KeyError if missing); `d.get(k)` -> None if missing, `d.get(k, default)`; `d[k]=v` adds OR **overwrites**; `d.pop(k)` / `del d[k]`; `.keys()` `.values()` `.items()` `.clear()` `.copy()`; `len(d)`. `x in d` **tests KEYS**, not values. `for k in d` loops keys; `for k,v in d.items()`. Keys unique & must be immutable (int, str, tuple OK; **list as key -> TypeError**); values anything. Lookup is very fast.

```
d = {'apples':450}; d['pears'] = 100
d['apples'] + d['pears']  -> 550
play['rock'] -> chained: m[p[p['rock']]]
def keys_with(x, ds):  # keys with value x
    return [k for k,v in ds.items() if v == x]
```

10 ALIASING, MUTABLE, COPY (Wk4a/4b)

- int/float/bool/str/tuple are **immutable**: `b = a; a = 5` -> b unchanged (copy of value). Lists/dicts/sets are **mutable**: `b = a` makes both names point to the SAME object (**alias**).

```
a=[1,2,3]; b=a; b[0]=99  # a=[99,2,3]!
c = a[:]  /  a.copy()  /  list(a)  # new
la=[9,8,7]; lb=[la,la]; lb[0][0]=0
 -> lb=[[0,8,7],[0,8,7]]  (same la twice)
[[0]*2]*2  -> two SAME inner lists
```

- **Shallow copy** (`[:]`, `.copy()`) copies only the outer list; inner lists are still shared. **Deep copy** copies nested lists too (recursive `deepcopy`).
- `L += x` modifies in place (aliases see it). `L = L + x` makes a NEW list (aliases do not change).

```
t1=['a','b','c']; t2=(t1,t1)
t1[1:]=['x','y','z']
t2 -> (['a','x','y','z'],['a','x','y','z'])
```

- `lst2, lst3 = lst, lst[:]`; `lst2.sort()` also sorts `lst` (alias) but `lst3` keeps the old order.
- **Passing to functions**: args are copies of the reference. Immutable arg -> caller never changes. List arg -> `lst.append(9)` / `lst[0]=9` **changes caller's list** (function 'modifies input', often returns None). But `lst = lst + [9]` or `lst = [..]` inside only rebinds the LOCAL name -> caller unchanged.
- Two styles: (1) **returns new** (`return tup + (x,)`): preserves input, must use result, slower; (2) **modifies in place** (`lst.append(x)`): faster, may return nothing.
- Speed: tuple rebuilt each time copies all items (6+7+8+... = ~5 billion for 100000 rounds); list append adds 1 item (~100000). 100000 runs: list 0.019 s vs tuple/new list ~21 s.

11 FILE I/O (Wk6a)

```
with open('f.txt') as f:     # 'r' default
    data = f.read()      # whole file: 1 str
with open('out.txt','w') as f:  # overwrite
    f.write('hi\n')          # str only; add \n
# 'a' = append at end; 'r' needs file to exist
```

- `with` auto-closes the file. `f.write()` needs a **str** (`str(5)`) and does NOT add newline. `'w'` erases existing content. Text mode is default (binary not needed).
- `f.readline()` reads ONE line **including** `\n` ('' at end of file). `f.read()` whole file (bad for big files). **File is iterable**: `for line in f:` reads line by line (best way, saves memory).
- Each line ends with `\n`: `print(line)` shows extra blank line. Remove with `line.rstrip('\n')` (rstrip strips right side; `strip()` both ends).
- `'a b\nc'.split()` -> ['a','b','c'] (default splits on spaces AND newlines). `line.split(',')` for CSV. Pieces are **str**: use `int()`/`float()` before maths.

```
with open('crude.csv') as f:
    f.readline()          # discard header
    year, rate = [], []
    for line in f:
        p = line.rstrip('\n').split(',')
        year.append(int(p[0]))
        rate.append(float(p[2])*1000)
```

- Marks file example: `data.split()[1::2]` = every 2nd item from index 1, then `[int(x) for x in ...]`. Don't 'hard-code' data in code; keep data in a file. `csv.reader(f)` also exists. Plot: `plt.plot(x,y)`, `plt.show()`.

12 HIGHER-ORDER FUNCTIONS, LAMBDA (Wk6b)

- Functions are **values**. `x = f` (no parens) stores the function; `x = f()` stores its result. `type(f)` is function. Normal variables aren't callable. Can be stored in lists: `[cos, len][1]([1,2,3])`=3.
- A function can take a function as input AND return a function. Nested `def` is local to outer function.

```
def make_power(n):
    return lambda x: x**n  # returns function
cube = make_power(3);   cube(2) -> 8
make_power(3)(2) -> 8   # call right away
def compose(f, g): return lambda x: f(g(x))
```

- **lambda**: `lambda args: expression`; one expression, **no** `return` **keyword** (value is the expression). `(lambda x: x**2)(2)`=4; `(lambda i,j: i**2+j**2)(3,4)`=25. Same as `def square(x): return x**2` but no name needed.

```
# lambda returning lambda (exam style)
(lambda p: lambda q: p(p(q)))(
  lambda x: x%10)(1432)   # = 2
 # p = x%10; p(p(1432)) = p(2) = 2
def perimeter(sides): return lambda l: l*sides
perimeter(5)(28) -> 140     # two () pairs
```

- Read nested calls inside-out: `f(a)(b)` = call f(a) first, then call the returned function on b. Extra `()` on a non-function -> TypeError.

```
def deriv(f):       # function in, function out
    dx = 1e-5
    return lambda x: (f(x+dx) - f(x)) / dx
# Newton (g(x)=0): x = x - g(x)/g'(x)
# repeat until abs(g(x)) < tiny; return x
# sqrt(A): g = lambda x: x*x - A
```

13 map / filter / DEEP OPERATIONS (Wk7)

- `map(f, seq)` applies f to every item; `filter(pred, seq)` keeps items where `pred(x)` is True (predicate returns True/False). Both return **iterables** (map/filter objects), not lists: wrap `list(...)`, `tuple(...)`.

```
list(map(lambda x: x*2, [1,2,3]))   # [2,4,6]
list(filter(lambda x: x%2==0, range(6)))
                                    # [0,2,4]
list(map(str, (1,2)))   # ['1','2'] tuple ok
```

TRAP: A map/filter object is **used up after one pass**: converting twice gives empty the 2nd time (`m=map(..); list(m); list(m)` -> `[]`). Store the result once, e.g. `L = list(m)`.

- Same map idea in comprehension: `[f(x) for x in s if cond]`. Lecture's own `map` only handled lists; Python's works on any sequence.
- **Shallow vs deep**: `len(s)` counts top-level items; `deepcount` counts leaves in nested lists. Test list-ness: `type(x) == list` or `isinstance(x, list)`.

```
def deepcount(s):
    if s == []: return 0
    if type(s) != list: return 1       # leaf
    return deepcount(s[0]) + deepcount(s[1:])
deepcount([[[1,2],3],9,[1,2]]) -> 6
def deepmap(f, s):   # deepsquare: f=x*x
    if s == []: return []
    if type(s) != list: return f(s)    # leaf
    return [deepmap(f,s[0])] + deepmap(f,s[1:])
def flatten(s):  # no nesting: [[[1]]] -> [1]
    if s == []: return []
    if type(s) != list: return [s]
    return flatten(s[0]) + flatten(s[1:])
```

TRAP: In deepmap the first item is wrapped in `[ ]` to **keep the nesting layer**; flatten does NOT wrap (that is why it removes nesting). Leaf case returns the value (or `[value]` in flatten), base returns `[]` / 0.

- Deep copy = deepmap with identity. Uses: directories, skeleton animation, images (list of rows of pixels of RGB lists).

14 EXAM CHECKLIST (from past papers)

- Papers (1 hr, ~30 MCQ + fill-blanks): expression eval, slicing, truthiness, `if/elif` tracing, loops with break/continue, tuple/list/set/dict outputs, aliasing, lambda, scope, call counting in recursion, fill-in code (count vowels, `grade` ladder, filter list, partition ints/strs, deep_convert).
- Always ask: **Is it mutable? Alias or copy? Does it return or print? Does slice or index?** `'e' in 'hello'` vs `in dict` (keys). Check if code errors: str+int, list key in dict, tuple assign, `remove` absent, missing key `[]`, IndexError, ValueError on int('x'), wrong arg count.
- Fill-in patterns: loop `for c in s:`, test `c in 'aeiou'`, `count += 1`; filter `for i in lst: if '4' not in str(i): out.append(i)`; type test `type(x) == int`; recursion `if seq == []: return []`; `isinstance(x,list)`.

15 VERIFIED QUICK ANSWERS (self-check)

```
10//8+6*4-2 -> 23
False and (1/0)>10 -> False
'qrstuvw'[3:4] -> 't'   'abc'[3:5] -> ''
'axolotl'[-4] -> 'l'
('x'*4+'y'*3)*2 -> 'xxxxyyyxxxxyyy'
[[1,3,5][1:]]*2+[7,9] -> [[3,5],[3,5],7,9]
len({5,3,8,5,9,1,8,5,9,3}) -> 5
set([3,3,3,3]) -> {3}
8 in {0:0,4:8} -> False (keys only)
{1:1,2:3}.get(4) -> None
d[[8]]=9 -> TypeError
tuple(i for i in range(0,5,-1)) -> ()
'abcdefg'[1:9] -> 'bcdefg'
```

- Set ops: `set('bananas') & set('pyjamas')` = {'a','s'}; `^` = letters in exactly one word. Decide membership with `in` before `append`/`remove` to avoid duplicates/ValueError.
- Tuple of lists: `tup=([1,2,3],4); tup[0][0]=999` works -> `([999,2,3],4)`; `tup[1]=0` fails (TypeError).
- Dict chain: follow lookups from the inside out, one `[ ]` at a time; each step is a key -> value.
- `print(f"{a}x")` f-strings insert variables inside `{ }`; `price[2]*2` is str repeat ('5'*2='55'), not maths.